SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I S.Mohammad Fuzail / 192565050 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

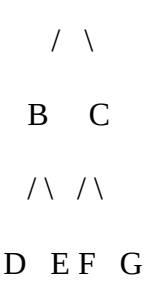
Signature of the student: Mohammad Fuzail

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	(empty)	G

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

Answer 1: Breadth-First Search (BFS) Dry Run

1. Understanding the Problem

The given graph is a binary tree with seven nodes. Node A is the root, B and C are its children, D and E are the children of B, and F and G are the children of C. In the form used by search algorithms, the graph can be written as an adjacency list, with the neighbours listed from left to right:

```
            A
          /   \
        B       C
       / \     / \
      D   E F   G

Adjacency list:  A: [B, C]   B: [D, E]   C: [F, G]   D: []   E: []   F: []   G: []
```

Breadth-First Search (BFS) explores a graph level by level. It first visits the start node, then all nodes at distance 1, then all nodes at distance 2, and so on. To do this it uses a **queue (FIFO: First In, First Out)**: the node that was added to the queue first is the node that is removed and expanded first. The queue holds the **frontier**, that is, the nodes that have been discovered but not yet visited.

Assumptions used in this dry run. (1) The search starts at A and there is no particular goal, so the whole graph is traversed. (2) The children of a node are added to the queue in left-to-right order. (3) A node is marked visited when it is removed from the queue and expanded. (4) Because the graph is a tree, no node can be reached twice, so no repeated-state check is actually needed, but a visited set is still shown in the algorithm for general graphs.

2. BFS Algorithm

```
BFS(start):
    queue <- empty FIFO queue;  visited <- empty list
    enqueue(queue, start)
    while queue is not empty:
        node <- dequeue(queue)              # remove from the FRONT
        append node to visited              # 'visit' the node
        for each child of node (left to right):
            if child is not already visited or in the queue:
                enqueue(queue, child)       # add at the REAR
    return visited
```

The main steps in every iteration are therefore: remove the front node, visit it, and add its children at the rear. The algorithm stops when the queue is empty.

3. Completed Table (as requested in the question)

The table below shows, for each iteration, the node that is removed from the front of the queue and visited, the children added, and the contents of the queue after the iteration (front of the queue is on the left). The initial queue before iteration 1 is [A].

Iteration	Queue before	Visited node (removed)	Children added	Queue after iteration
1	[A]	A	B, C	[B, C]
2	[B, C]	B	D, E	[C, D, E]
3	[C, D, E]	C	F, G	[D, E, F, G]
4	[D, E, F, G]	D	none (leaf)	[E, F, G]
5	[E, F, G]	E	none (leaf)	[F, G]
6	[F, G]	F	none (leaf)	[G]
7	[G]	G	none (leaf)	[ ] (empty)

4. Step-by-Step Explanation of Each Iteration

- **Iteration 1:** The queue contains only the start node [A]. A is removed and visited. Its children B and C are added at the rear, so the queue becomes [B, C].
- **Iteration 2:** B is at the front, so it is removed and visited. Its children D and E are added behind C. The queue is [C, D, E]. Note that C, which is at the same level as B, is still ahead

of D and E, because it was added earlier. This is how BFS finishes a whole level before going deeper.

- **Iteration 3:** C is removed and visited. Its children F and G are added, giving [D, E, F, G]. This is the largest queue in the whole run (4 nodes), because the whole bottom level (level 2) is now waiting in the queue.
- **Iteration 4:** D is removed and visited. D is a leaf, so nothing is added. Queue: [E, F, G].
- **Iteration 5:** E is removed and visited. It is also a leaf. Queue: [F, G].
- **Iteration 6:** F is removed and visited (leaf). Queue: [G].
- **Iteration 7:** G is removed and visited (leaf). The queue is empty, so BFS stops. All 7 nodes have been visited exactly once.

5. Answers to the Questions

Question 1: BFS traversal order.

The nodes are visited in the order **A → B → C → D → E → F → G**. This is the same as reading the tree level by level from left to right: level 0 = A; level 1 = B, C; level 2 = D, E, F, G.

Question 2: Queue contents after each iteration.

After iteration	1	2	3	4	5	6	7
Queue (front → rear)	B, C	C, D, E	D, E, F, G	E, F, G	F, G	G	empty
Node just visited	A	B	C	D	E	F	G

The queue grows while the nodes of the upper levels are expanded (iterations 1 to 3) and then shrinks as the leaves are visited (iterations 4 to 7). It never needs to hold more than the number of nodes on one level, here 4.

6. Level-wise View and Queue Behaviour

Level (depth)	Nodes on the level	Visited in iterations	Distance from A
0	A	1	0 edges
1	B, C	2, 3	1 edge
2	D, E, F, G	4, 5, 6, 7	2 edges

Because the queue is first-in-first-out, every node of level k is visited before any node of level k + 1. This level-order property is the reason why BFS finds the **shortest path (in number of edges)** from the start node to any other node. For example, the shortest path from A to F is A → C → F, found when F is added to the queue in iteration 3 with its parent recorded as C.

Using BFS to find a path. If a goal node were given, BFS would store the parent of each node at the time it is added to the queue. Following the parents backwards from the goal to A and reversing the list gives the solution path. In this graph: parent(B) = A, parent(C) = A, parent(D) = B, parent(E) = B, parent(F) = C, parent(G) = C. For the goal G the path is A → C → G, of length 2.

7. Complexity and Properties

Property	Value for BFS	Explanation
Time complexity	O(V + E) on a graph, or O(b^d) on a tree	Every node is enqueued and dequeued once, and every edge is examined once. For this graph V = 7 and E = 6, so 13 basic steps.
Space complexity	O(b^d), the width of the widest level	The frontier can hold a whole level. Here the maximum queue size is 4.
Completeness	Yes	If a solution exists at a finite depth and the branching factor is finite, BFS will find it.
Optimality	Yes, for unit step costs	Shallower nodes are always expanded first, so the first time a goal is reached is along a shallowest path.
Data structure	Queue (FIFO)	Using a stack instead would change the algorithm into DFS.

Here b is the branching factor (2 in this tree) and d is the depth of the shallowest goal. The main weakness of BFS is memory: on larger problems the frontier grows exponentially with the depth, which is a practical limit on the sizes of problems that can be solved.

8. Variations and Verification

- **Order of children.** If the children were added from right to left, the traversal would be A, C, B, G, F, E, D. The level-by-level pattern is the same; only the order inside each level changes. This is why the question states the left-to-right assumption.
- **Checking the answer.** The traversal contains each of the 7 nodes exactly once. The number of iterations equals the number of nodes (7), which matches the 7 rows of the table. Total enqueue operations are also 7 (A at the start, plus 6 children), and total dequeue operations are 7.
- **Early goal test.** If a goal such as F were specified, the search could stop as soon as F is visited, which happens in iteration 6. If the goal test is applied when F is generated (iteration 3), the search could stop even earlier. Both give the same shortest path A → C → F.
- **Applications.** BFS is used for shortest paths in unweighted graphs, finding the minimum number of moves in puzzles, web crawling by link distance, friend suggestions in social networks, and as a basis of other algorithms such as Dijkstra's algorithm and the Ford–Fulkerson method with shortest augmenting paths.

9. Python Implementation (for Checking the Dry Run)

```
from collections import deque

graph = {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F', 'G'],
         'D': [], 'E': [], 'F': [], 'G': []}

def bfs(start):
    queue = deque([start])          # FIFO queue
    order = []
    while queue:
        node = queue.popleft()      # remove from the front
        order.append(node)          # visit
```

```
        for child in graph[node]:   # left to right
            queue.append(child)     # add at the rear
        print(len(order), node, list(queue))
    return order

print(bfs('A'))
# Output lines:  1 A ['B', 'C']   2 B ['C', 'D', 'E']   3 C ['D', 'E', 'F', 'G']
#                4 D ['E', 'F', 'G']   5 E ['F', 'G']   6 F ['G']   7 G []
# Final result:  ['A', 'B', 'C', 'D', 'E', 'F', 'G']
```

The program prints exactly the visited node and the queue contents of the table in section 3, which confirms the hand dry run.

10. Worked Example: BFS with a Goal Node and Parent Links

To show how a solution path is obtained, suppose the goal is node G. BFS records the parent of every node when it is placed in the queue and stops when the goal is removed from the queue.

Iteration	Node removed	Is it the goal?	Parent links recorded in this iteration	Queue after
1	A	No	B ← A, C ← A	[B, C]
2	B	No	D ← B, E ← B	[C, D, E]
3	C	No	F ← C, G ← C	[D, E, F, G]
4	D	No	none	[E, F, G]
5	E	No	none	[F, G]
6	F	No	none	[G]
7	G	Yes, stop	none	[ ]

Tracing the parents backwards gives G ← C ← A, so the solution path is **A → C → G** with cost 2. BFS needed 7 iterations to remove G from the queue, but the path itself is short (2 edges). The search effort depends on the breadth of the tree, whereas the path length depends on the depth of the goal.

Goal node	Iteration in which BFS visits it	Shortest path found	Path length (edges)
B	2	A → B	1
C	3	A → C	1
D	4	A → B → D	2
E	5	A → B → E	2
F	6	A → C → F	2
G	7	A → C → G	2

11. Size of the Queue during the Run

After iteration	1	2	3	4	5	6	7
Queue length	2	3	4	3	2	1	0
Nodes discovered so far (visited + queued)	3	5	7	7	7	7	7

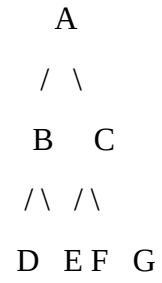
The queue length rises to a maximum of 4 at iteration 3 and then falls to 0. This number is the memory used by the search at its peak. In this small tree it is tiny, but in a tree with branching factor b the queue can hold about b^d nodes at depth d, which is why BFS is expensive for deep searches.

12. Checklist for Writing a Correct BFS Dry Run

- Write the start node in the queue before iteration 1.
- In every iteration, remove only the **front** node, mark it as visited and add its children at the **rear**.
- Follow the stated order of children (left to right here) and do not add a node that has already been visited or is already in the queue.
- Write the queue contents after each iteration, with the front clearly marked.
- Stop when the queue is empty (or when the goal is removed, if there is a goal).
- Finally check that every node appears exactly once in the traversal order.

Conclusion. The BFS dry run on the given tree produces the traversal order A, B, C, D, E, F, G, with the queue changing as shown in the table. The dry run confirms the key features of BFS: it uses a FIFO queue, it explores level by level, and it visits each node once.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

Answer 2: Depth-First Search (DFS) Dry Run

1. Understanding the Problem

The same binary tree is used again, with A as the root, B and C as its children, D and E below B, and F and G below C. This time the graph is traversed with **Depth-First Search (DFS)**. DFS goes as deep as possible along one branch before returning (backtracking) and trying the next branch. It uses a **stack (LIFO: Last In, First Out)**, either explicitly or through the recursion stack of the programming language.

```
            A
          /   \
         B     C
        / \   / \
       D   E F   G

Adjacency list:  A: [B, C]   B: [D, E]   C: [F, G]   D: []   E: []   F: []   G: []
```

Assumptions. (1) The start node is A. (2) The children must be explored from left to right, so B is explored before C, D before E, and F before G. Because a stack removes the most recently added node first, the children are **pushed in reverse order** (right child first, then left child) so that the left child ends on top of the stack. (3) The stack is shown with its **top on the left**. (4) A node is visited when it is popped from the stack.

2. DFS Algorithm

```
DFS(start):                        # iterative version with an explicit stack
    stack <- empty stack;  visited <- empty list
    push(stack, start)
    while stack is not empty:
        node <- pop(stack)                    # remove from the TOP
        if node is not in visited:
            append node to visited            # 'visit' the node
            for each child of node (right to left):
                push(stack, child)            # left child ends on top
    return visited

DFS-Recursive(node):
    mark node visited
    for each child of node (left to right):
        if child not visited: DFS-Recursive(child)
```

The recursive version does the same work, because every recursive call is an entry on the system call stack. It is shorter, but for very deep graphs the explicit stack avoids stack-overflow errors.

3. Completed Table (Iteration, Stack, Visited Node)

The table lists, for each iteration, the node that is popped and visited, the children pushed, and the stack after the iteration (top of the stack on the left). The stack before iteration 1 is [A].

Iteration	Stack before (top → bottom)	Visited node (popped)	Children pushed	Stack after (top → bottom)
1	[A]	A	C, then B	[B, C]
2	[B, C]	B	E, then D	[D, E, C]
3	[D, E, C]	D	none (leaf)	[E, C]
4	[E, C]	E	none (leaf)	[C]
5	[C]	C	G, then F	[F, G]
6	[F, G]	F	none (leaf)	[G]
7	[G]	G	none (leaf)	[ ] (empty)

Traversal order obtained: A → B → D → E → C → F → G.

4. Step-by-Step Explanation

- **Iteration 1:** A is popped and visited. The children are pushed in reverse order: C first and then B, so B is on top. Stack: [B, C].
- **Iteration 2:** B (top) is popped and visited. E and then D are pushed, so D is on top. Stack: [D, E, C]. The search has now gone one level deeper in the left branch, and C (the right branch of A) waits at the bottom of the stack.
- **Iteration 3:** D is popped and visited. It is a leaf, so nothing is pushed. This is the point where DFS has reached the bottom of the left branch. Stack: [E, C].
- **Iteration 4:** E is popped and visited. It is also a leaf. This completes the whole left subtree of A (B, D and E). Stack: [C]. Here DFS has 'backtracked' to the next unexplored branch, in this case C.
- **Iteration 5:** C is popped and visited. G and then F are pushed, so F is on top. Stack: [F, G].
- **Iteration 6:** F is popped and visited (leaf). Stack: [G].
- **Iteration 7:** G is popped and visited (leaf). The stack is empty, so the search ends with all 7 nodes visited once.

5. Discovery and Finishing Times (Recursive View)

In the recursive version each node gets a discovery time (when the call starts) and a finishing time (when the call ends). A single counter increases at each event. The intervals show how the calls are nested: a child's interval always lies inside its parent's interval.

Node	Discovery time	Finishing time	Parent	Remark
A	1	14	none (root)	Starts first, finishes last

Node	Discovery time	Finishing time	Parent	Remark
B	2	7	A	Finishes after D and E
D	3	4	B	Leaf
E	5	6	B	Leaf
C	8	13	A	Started only after B's whole subtree is finished
F	9	10	C	Leaf
G	11	12	C	Leaf

Sorting the nodes by discovery time gives the same order as the iterative version: A, B, D, E, C, F, G. All edges of this graph are **tree edges**; since it is a tree there are no back, forward or cross edges.

6. Answers in Summary

- **DFS traversal order:** A, B, D, E, C, F, G.
- **Stack contents after each iteration:** [B, C]; [D, E, C]; [E, C]; [C]; [F, G]; [G]; [ ] (top on the left).
- **Maximum stack size:** 3 (after iteration 2), compared with a maximum queue size of 4 for BFS on the same graph.
- **Total visited nodes:** 7, each exactly once.

7. Comparison of DFS with BFS on the Same Graph

Feature	BFS	DFS
Data structure	Queue (FIFO)	Stack (LIFO) or recursion
Order of visiting	A, B, C, D, E, F, G (level by level)	A, B, D, E, C, F, G (branch by branch)
Maximum frontier size here	4 nodes	3 nodes
Memory growth	O(b^d): whole level is stored	O(b·m): only one path and its siblings are stored
Finds shortest path?	Yes (unit costs)	No, the first path found may be longer
Complete?	Yes	Only if the graph is finite or repeated states are checked; may loop forever on infinite or cyclic graphs without a visited check
Typical uses	Shortest path, minimum moves, level-order tasks	Cycle detection, topological sort, connected components, maze solving, backtracking

Here m is the maximum depth of the graph. DFS needs much less memory than BFS on deep and wide trees because it stores only the nodes along the current path together with their unexplored siblings. The price is that it may find a deeper, more expensive goal first, and it can follow an infinitely long branch if the search space is infinite.

8. Variations and Verification

- **Order of pushing.** If the children were pushed left to right (B then C, and so on) without reversing, the right child would be on top and the traversal would become A, C, G, F, B, E, D. The result is also a valid DFS order, but it explores the right branch first, which is why the push order must be stated.
- **Checking the answer.** The traversal contains 7 distinct nodes. Each node is pushed exactly once and popped exactly once (7 pushes in total counting A, and 7 pops). In the tree, the DFS order is the same as the **pre-order traversal** (root, left subtree, right subtree).
- **Goal search.** If the goal were F, DFS would stop in iteration 6 and the path A → C → F could be recovered from the parent links. If the goal were E, DFS would stop at iteration 4 with the path A → B → E.
- **Depth-limited and iterative-deepening DFS.** To avoid the weakness of plain DFS in deep or infinite spaces, a depth limit can be used. Iterative-deepening DFS repeats depth-limited DFS with limits 0, 1, 2, … and combines the low memory of DFS with the completeness and optimality of BFS.

9. Python Implementation (for Checking the Dry Run)

```
graph = {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F', 'G'],
         'D': [], 'E': [], 'F': [], 'G': []}

def dfs_iterative(start):
    stack = [start]                          # last element = top of the stack
    order = []
    while stack:
        node = stack.pop()                   # remove from the top
        order.append(node)                   # visit
        for child in reversed(graph[node]): # push right child first
            stack.append(child)
        print(len(order), node, stack[::-1])   # print stack with top on the left
    return order

def dfs_recursive(node, order=None):
    order = [] if order is None else order
    order.append(node)
    for child in graph[node]:
        dfs_recursive(child, order)
    return order

# Both return ['A', 'B', 'D', 'E', 'C', 'F', 'G']
```

The printed stacks are [B, C], [D, E, C], [E, C], [C], [F, G], [G] and [ ], which are the same as in the completed table, so the hand dry run is confirmed.

10. Worked Example: DFS with a Goal Node and Parent Links

Suppose the goal is node E. DFS records the parent of each node when it is pushed and stops when the goal is popped.

Iteration	Node popped	Is it the goal?	Parent links recorded	Stack after (top first)
1	A	No	B ← A, C ← A	[B, C]
2	B	No	D ← B, E ← B	[D, E, C]
3	D	No	none	[E, C]

Iteration	Node popped	Is it the goal?	Parent links recorded	Stack after (top first)
4	E	Yes, stop	none	[C]

Following the parents backwards gives E ← B ← A, so the path is **A → B → E**. DFS found the goal after only 4 iterations, while BFS needed 5 iterations to reach E. For a goal on the right-hand side the situation is reversed, as the following table shows.

Goal node	Iteration in which DFS visits it	Iteration in which BFS visits it	Which is faster?
B	2	2	Equal
C	5	3	BFS
D	3	4	DFS
E	4	5	DFS
F	6	6	Equal
G	7	7	Equal

The comparison shows that neither method is always faster. DFS is quick when the goal lies deep on the branch that it explores first, and BFS is quick when the goal is near the root. Note that for goal C the DFS path is still A → C (cost 1), but DFS had to finish the whole left subtree first; in a graph with longer branches, DFS could even return a longer, non-optimal path.

11. Size of the Stack during the Run

After iteration	1	2	3	4	5	6	7
Stack size	2	3	2	1	2	1	0
Stack contents (top first)	B, C	D, E, C	E, C	C	F, G	G	empty

The stack never holds more than 3 nodes: the current path from the root plus the unexplored siblings of the nodes on that path. The maximum stack size grows only linearly with the depth of the tree (about b·m nodes), whereas the BFS queue can grow exponentially, which is the main memory advantage of DFS.

12. Checklist for Writing a Correct DFS Dry Run

- Put the start node on the stack before iteration 1 and decide which side is the top (here: the left side of the written stack).
- In every iteration pop the **top** node, mark it visited and push its children in **reverse** order so that the left child will be explored first.
- Do not visit a node twice; in a general graph check whether the node is already visited.
- Write the stack contents after every iteration, showing clearly which end is the top.
- Stop when the stack is empty (or when the goal is popped).
- Check that all nodes appear exactly once and that the result is a valid pre-order of the tree.

Conclusion. The DFS dry run on the given tree gives the traversal order A, B, D, E, C, F, G, with the stack changing as shown in the table. It illustrates the main ideas of DFS: it uses a LIFO stack, it explores one branch completely before backtracking, and it needs less memory than BFS.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :
1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	S0 = 136/502/478	[S0]	[S1, S2, S3, S4]
2	S1 = 106/532/478	[S1, S2, S3, S4]	[S2, S3, S4, S5, S6]
3	S2 = 136/572/408	[S2, S3, S4, S5, S6]	[S3, S4, S5, S6, S7, S8]
4	S3 = 136/052/478	[S3, S4, S5, S6, S7, S8]	[S4, S5, S6, S7, S8, S9, S10]
5	S4 = 136/520/478	[S4, S5, S6, S7, S8, S9, S10]	[S5, S6, S7, S8, S9, S10, S11, S12]

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

Answer 3: BFS Dry Run on the 8-Puzzle

1. Understanding the Problem

The 8-puzzle is played on a 3 × 3 board with eight numbered tiles and one empty square (the blank, written as 0 or _). A move slides a tile next to the blank into the blank's position, which is the same as moving the blank up, down, left or right. The task is to find a sequence of moves that changes the **initial state** into the **goal state**:

```
Initial state            Goal state
  1  3  6                  1  2  3
  5  _  2                  4  5  6
  4  7  8                  7  8  _
```

Search problem formulation.

- **State:** a placement of the tiles 1–8 and the blank on the 9 squares. Written compactly row by row, e.g. the initial state is 136/502/478.

- **Initial state:** 136/502/478. **Goal state:** 123/456/780.
- **Actions (operators):** move the blank Up, Down, Left or Right, if the move stays on the board. The number of legal actions is 2 (blank in a corner), 3 (blank on an edge) or 4 (blank in the centre).
- **Transition model:** the blank exchanges places with the neighbouring tile.
- **Step cost:** 1 for every move, so the path cost equals the number of moves.
- **Solvability check:** the number of inversions of the tile sequence 1, 3, 6, 5, 2, 4, 7, 8 (ignoring the blank) is 6 (3>2; 6>5, 6>2, 6>4; 5>2, 5>4). For a 3 × 3 board a puzzle is solvable when this number is even, and the goal has 0 inversions, so this puzzle **is solvable**.

Assumptions for the dry run. (1) The operators are applied in the fixed order **Up, Down, Left, Right** (movement of the blank). (2) BFS is the *graph-search* version: a state that has already been generated is not added to the queue again. (3) The goal test is applied when a state is removed from the queue for expansion. (4) States are named **S0, S1, S2, …** in the order in which they are generated, so S0 is the initial state. A different operator order would give a different numbering and a different count of nodes in the last level, but the same solution length.

2. BFS Algorithm for the 8-Puzzle

```
BFS-8-PUZZLE(initial, goal):
    queue <- [initial];  generated <- {initial}
    while queue is not empty:
        state <- dequeue(queue)                  # front of the FIFO queue
        if state = goal: return path(state)      # follow parent links back to the
start
        for each move in (Up, Down, Left, Right) legal for the blank:
            child <- result of the move
            if child not in generated:
                parent(child) <- state;  add child to generated;  enqueue(queue,
child)
    return failure
```

3. Completed Table (First Five Iterations)

The table in the question is completed in the same way. In each iteration the front state of the queue is expanded, i.e. all its legal successors are generated and added to the rear of the queue. Detailed trace of the first iterations (queue contents are given with the front on the left):

Iteration	Current state	Queue before expansion	Queue after expansion
1	S0 = 136/502/478	[S0]	[S1, S2, S3, S4]
2	S1 = 106/532/478	[S1, S2, S3, S4]	[S2, S3, S4, S5, S6]
3	S2 = 136/572/408	[S2, S3, S4, S5, S6]	[S3, S4, S5, S6, S7, S8]
4	S3 = 136/052/478	[S3, S4, S5, S6, S7, S8]	[S4, S5, S6, S7, S8, S9, S10]
5	S4 = 136/520/478	[S4, S5, S6, S7, S8, S9, S10]	[S5, S6, S7, S8, S9, S10, S11, S12]

Key to the state names (the board is given row by row; 0 is the blank):

State	Board	Generated from	Depth
S0	136/502/478	start	0

State	Board	Generated from	Depth
S1	106/532/478	S0 + Up	1
S2	136/572/408	S0 + Down	1
S3	136/052/478	S0 + Left	1
S4	136/520/478	S0 + Right	1
S5	016/532/478	S1 + Left	2
S6	160/532/478	S1 + Right	2
S7	136/572/048	S2 + Left	2
S8	136/572/480	S2 + Right	2
S9	036/152/478	S3 + Up	2
S10	136/452/078	S3 + Down	2
S11	130/526/478	S4 + Up	2
S12	136/528/470	S4 + Down	2

Detailed trace of the first twelve iterations (the successors generated in each iteration, with the operator used):

Iteration	Expanded state	Depth	Successors generated (added to queue)
1	S0	depth 0	S1 (Up), S2 (Down), S3 (Left), S4 (Right)
2	S1	depth 1	S5 (Left), S6 (Right)
3	S2	depth 1	S7 (Left), S8 (Right)
4	S3	depth 1	S9 (Up), S10 (Down)
5	S4	depth 1	S11 (Up), S12 (Down)
6	S5	depth 2	S13 (Down)
7	S6	depth 2	S14 (Down)
8	S7	depth 2	S15 (Up)
9	S8	depth 2	S16 (Up)
10	S9	depth 2	S17 (Right)
11	S10	depth 2	S18 (Right)
12	S11	depth 2	S19 (Left)

Looking at the first iterations: the initial state S0 has the blank in the centre, so all four moves are legal and four children S1–S4 are created. S1 (blank moved up) can only go Left or Right, because going Down would return to S0, which has already been generated. In the same way S2, S3 and S4 each produce two new children. After iteration 5 all states of depth 1 have been expanded and the queue contains the 8 states of depth 2 (S5–S12) waiting in order.

4. Answers to the Questions

Question 1: Which node is expanded in each iteration?

The node at the front of the queue is expanded. The first five iterations expand **S0 (initial state), S1, S2, S3 and S4**, as shown in the table. After that the algorithm expands S5, S6, … in the same

order in which they were generated (first-in, first-out). In general, BFS expands all states of depth 0, then all states of depth 1, then depth 2, and so on. The goal is reached at depth 8, and in the order of expansion it is the **311th state removed from the queue**, so 310 states are expanded and then the goal is found. The goal test is applied to the 311th state.

Question 2: How many states are generated in total?

With the assumptions given above (duplicates are discarded, the goal test is done when a state is removed from the queue), BFS **generates 500 distinct states** including the initial state (499 generated successors plus the start), and **expands 310 states** before it removes the goal from the queue. The count by depth is:

Depth	States generated at this depth	States expanded at this depth	Total generated so far
0	1	1	1
1	4	4	5
2	8	8	13
3	8	8	21
4	16	16	37
5	32	32	69
6	60	60	129
7	72	72	201
8	136	109	337
9	163	0	500

At depth 8, the level that contains the goal, only 109 of the 136 states had been expanded when the goal was removed. Expanding these produced the 163 states at depth 9. If the goal test is applied as soon as a state is generated (early goal test), the search stops earlier, after generating 311 states and expanding 187. The numbers therefore depend on the operator order and on when the goal test is done, but the solution depth (8) does not.

Question 3: Complete solution path from the initial state to the goal state.

The sequence of blank moves is **Right → Up → Left → Down → Left → Down → Right → Right** (8 moves). The states along the path are:

```
Initial              Step 1               Step 2
                     blank Right          blank Up
1 3 6                1 3 6                1 3 _
5 _ 2                5 2 _                5 2 6
4 7 8                4 7 8                4 7 8

Step 3               Step 4               Step 5
blank Left           blank Down           blank Left
1 _ 3                1 2 3                1 2 3
5 2 6                5 _ 6                _ 5 6
4 7 8                4 7 8                4 7 8

Step 6               Step 7               Step 8 (Goal)
blank Down           blank Right          blank Right
1 2 3                1 2 3                1 2 3
4 5 6                4 5 6                4 5 6
_ 7 8                7 _ 8                7 8 _
```

Step	Move of the blank	Board (row by row)	Path cost g
0	start	136/502/478	0
1	blank Right	136/520/478	1
2	blank Up	130/526/478	2
3	blank Left	103/526/478	3
4	blank Down	123/506/478	4
5	blank Left	123/056/478	5
6	blank Down	123/456/078	6
7	blank Right	123/456/708	7
8	blank Right	123/456/780	8

In terms of tiles, this is the same as sliding tile 2, then tile 6, then tile 3, then tile 2 again, then tile 5, tile 4, tile 7 and tile 8 into the blank square. The path ends in the goal state 123/456/780, and its cost is 8.

Question 4: Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

- **Every move costs the same.** In the unweighted 8-puzzle each move has cost 1, so the cost of a path is simply the number of moves, which equals the depth of the node in the search tree.
- **BFS expands nodes in order of non-decreasing depth.** Because the queue is FIFO, all nodes of depth d are removed from the queue before any node of depth d + 1. (Children are always added behind the existing nodes, and a child's depth is its parent's depth plus 1.)
- **Hence the first time the goal is reached it is at the smallest possible depth.** Suppose a shorter solution of depth k existed. Then its goal node would have been generated while expanding a node of depth k − 1 and removed from the queue before the deeper goal found. This contradicts the order of expansion. In this run, all 201 states of depth 0 to 7 were expanded, and none of them is the goal, so no solution with 7 or fewer moves exists.
- **Completeness.** The branching factor is finite (at most 4) and the state space is finite (181,440 reachable states), so BFS cannot miss a reachable goal.
- **Contrast with other algorithms.** DFS could find a much longer solution first, and may go very deep along a branch that never reaches the goal. Uniform-cost search gives the same optimal result as BFS here, since all costs are equal, but with more overhead.

The statement is true only because the step costs are equal. If moves had different costs (a weighted problem), BFS would minimise the number of moves but not the total cost, and uniform-cost search or A* would have to be used.

5. Analysis of the Search Effort

Measure	Value in this dry run	Comment
Solution depth d	8 moves	Optimal, as shown by BFS

Measure	Value in this dry run	Comment
Branching factor b	2 to 4 legal moves per state	Depends on the blank's position; repeated states are removed, so fewer new children appear
States generated	500	Includes the initial state; goal test on removal
States expanded	310	The goal is the 311th state removed
Maximum queue length	190 states	Shows the memory demand of BFS
Size of the whole reachable space	181,440 states (9!/2)	BFS here explores only about 0.3% of it
Time and space complexity	O(b^d)	Both grow exponentially with the solution depth

The table shows the central trade-off of BFS: it guarantees the best solution, but the number of states stored in memory grows quickly with the depth of the solution. For a solution depth of 8 this is manageable. For hard 8-puzzle instances (depth up to 31) and especially for the 15-puzzle, plain BFS becomes impractical, and informed methods such as A* with the Manhattan-distance or misplaced-tiles heuristic are used instead, which expand far fewer states while still returning an optimal solution.

6. Verification and Common Mistakes

- **Replay the path.** Starting from 136/502/478 and applying Right, Up, Left, Down, Left, Down, Right, Right in order, each board in the table is obtained from the previous one by a single legal blank move, and the last board is the goal. This confirms that the path is valid.
- **Parity check.** Every move shifts the blank to a neighbouring square, which changes its chessboard colour. The blank starts in the centre square and ends in the bottom-right corner, and these two squares have the same colour, so the number of moves must be even. The solution length 8 is even, which is consistent.
- **Do not re-add states.** A common error is to put the parent state back into the queue (for example S0 as a child of S1). The generated-set avoids this, which is why S1 has only two children.
- **Keep the queue order.** Another frequent error is to put the new states in front of the queue (LIFO), which turns the algorithm into DFS and loses the guarantee of the shortest solution.
- **State the operator order.** The labelling of states and the exact numbers of generated and expanded states depend on the order of the operators, so the order must always be written down in the dry run.

Conclusion. BFS applied to the 8-puzzle from 136/502/478 reaches the goal 123/456/780 in **8 moves** (Right, Up, Left, Down, Left, Down, Right, Right). With the stated assumptions it generates 500 states and expands 310. Because BFS expands states level by level and every move costs 1, the solution found is guaranteed to be the shortest possible.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.